

PTP360 · Second-Pass QA

Deployment guide — going from local dev to a live URL

0. Read this first: there is only one app to deploy

This project is a single Next.js (App Router) application. The pages you click through (Coverage Check, Evidence Consistency, Contradiction Flag) and the **/api/*** routes that call Gemini live in the **same codebase and the same deployment**. Next.js API routes already run server-side, so your Gemini keys are read from environment variables on the server and are **never** sent to the browser — that part is already correct in the code.

That means the plan of "frontend on Vercel, backend on Render (or similar) for the API keys" isn't quite the right mental model here — there's no separate backend to build. Whichever platform hosts this Next.js app *is* the backend. The recommendation below is to deploy everything to **Vercel as one project**. A section at the end covers Render as an alternative, in case you have a reason to prefer it, but it replaces Vercel rather than sitting behind it.

Good to know: Splitting a Next.js app into a separate frontend + backend deployment is normally more work, not less — you'd need to add CORS handling to every API route, duplicate environment variables across two dashboards, and manage two deploy pipelines instead of one. Skip it unless a specific requirement forces it.

1. Yes — run npm run build locally first

Vercel and Render both run their own build on their own infrastructure, so this step isn't strictly required for them to work. But it's the fastest way to catch a broken build on your own machine, in seconds, instead of waiting on a remote build log. This was already run against the current code and it succeeds cleanly:

```
npm run build
```

Expected output (already verified for this codebase):

```
Route (app)
[static] /
[static] /_not-found
[fn] /api/check-room
[fn] /api/contradiction-check
[fn] /api/coverage-check
[fn] /api/evidence-check
[fn] /api/export-gap-report
[fn] /api/export-report
[fn] /api/room-walkthrough-check
[fn] /api/upload-report
[static] /contradiction-check
[static] /coverage
[static] /evidence-check

[static] prerendered as static content
```

```
[fn] server-rendered on demand (serverless function)
```

If **npm run build** fails on your machine, it will fail on Vercel/Render too — fix it before pushing. If it succeeds locally but fails only on the host, that's almost always a Node version mismatch (see the troubleshooting table at the end).

Recommended: This project's **package.json** has no "engines" field pinning a Node version. Next.js 16 requires Node ≥ 20.9 . Most hosts default to a recent LTS already, but if you hit odd build errors on the host only, add this to package.json and redeploy:

```
"engines": {  
  "node": ">=20.9.0"  
}
```

2. Recommended: deploy everything to Vercel

One deployment. Vercel builds the pages as static/prerendered content and turns every file under **app/api/*route.ts** into its own serverless function automatically — that's your "backend," already handled.

Step 1 — Push to GitHub

If the repo isn't already on GitHub, push it there. Confirm your API keys are never committed:

```
git status
cat .gitignore # should list .env
```

This repo's .env file already holds your 3 real keys and is expected to be gitignored — double-check it never shows up in "git status" as a file to be committed.

Step 2 — Import the project

- 1 Go to vercel.com and sign in (GitHub login is simplest).
- 2 Click **New Project** → select this repository → **Import**.
- 3 Framework Preset: Vercel auto-detects **Next.js**. Leave the Build Command (`next build`) and Output Directory on their defaults — don't override them.

Step 3 — Set environment variables

Before clicking Deploy (or right after, then redeploy), go to **Project Settings** → **Environment Variables** and add each of the following for the **Production** environment (add them for Preview too if you want pull-request preview deployments to actually work):

Variable	Used by	Example / notes
GEMINI_API_KEY_1	Coverage Check (coverage-check, check-room, upload-report)	Your Feature 1 key from AI Studio
GEMINI_API_KEY_2	Evidence Consistency (evidence-check, room-walkthrough-check)	Your Feature 2 key
GEMINI_API_KEY_3	Contradiction Flag (contradiction-check)	Your Feature 3 key
GEMINI_MODEL	All three features	e.g. gemini-3.5-flash-lite
NEXT_PUBLIC_APP_URL	Not read anywhere in the code yet — optional	Safe to skip for now

Good to know: Never prefix these with **NEXT_PUBLIC_**. That prefix tells Next.js to bake the value into the JavaScript sent to the browser — the opposite of what you want for a secret key. The code already reads them as plain server-side variables, which is correct; just don't rename them in a way that adds that prefix.

Step 4 — Deploy

Click **Deploy**. Vercel runs next `build` on its own infrastructure and gives you a live `*.vercel.app` URL a minute or two later.

Step 5 — Smoke-test the live URL

Click through all three tabs on the *deployed* site, not just locally. This specifically exercises the parts of the app that read local files at request time (`data/checklist.json`, `data/sample-tour.json`, `public/sample-images/*`) rather than importing them statically:

- **Coverage Check** → Load demo data, Upload report (1 file), Download report
- **Evidence Consistency** → Single Photo mode with 1 image, then switch to Room Walkthrough mode with 1 image
- **Contradiction Flag** → pick a sample pair, Check agreement

Vercel's build tracer (the same `@vercel/nft` engine Next.js uses for its own output: `"standalone"` mode) statically detects `fs.readFile(path.join(process.cwd(), "data", ...))`-style calls and bundles those files automatically — this codebase already uses that exact pattern, so it should work without extra config. If any of the file-reading routes 500 on the live site but work locally, see the troubleshooting table at the end for the one-line fix.

Step 6 — (Optional) function timeout for slower calls

Room Walkthrough mode sends up to 6 images in a single Gemini call, and Coverage Check's "Load demo data" fires several calls in parallel. If you see a timeout on a slower request, add this export to the relevant route file and redeploy (check your current Vercel plan's maximum allowed value first — it differs between plans):

```
export const maxDuration = 60; // seconds
```

Step 7 — Redeploys

Every push to your default branch (usually `main`) auto-redeploys. Changing an environment variable does **not** retroactively affect an already-running deployment — Vercel will prompt you to redeploy after saving new values; do that.

3. Alternative: skip Vercel and use Render instead

If you'd still rather use Render — for example, to keep everything on one platform you already use for other projects — deploy the **whole app** there as a single Node web service. This replaces Vercel; it isn't a second deployment alongside it.

- 1 On Render: **New** → **Web Service** → connect this GitHub repo.
- 2 Environment: **Node**.
- 3 Build Command: `npm install && npm run build`
- 4 Start Command: `npm run start` (this runs `next start`, which serves both the pages and the API routes from one process)
- 5 Add the same environment variables as the Vercel table above (`GEMINI_API_KEY_1/2/3`, `GEMINI_MODEL`) under Render's Environment tab.
- 6 Set the Node version explicitly (Render dashboard → Environment, or a `.node-version` file in the repo root containing `20.9.0` or newer) — Next.js 16 requires `≥ 20.9`.
- 7 Deploy, then run the exact same Step 5 smoke test from the Vercel section above against your Render URL.

Good to know: Render's free tier spins the service down after inactivity; the first request after a cold start can take significantly longer while it wakes up and while Next.js's server starts. That's a Render platform behavior, not something in this codebase — a paid "always-on" tier avoids it if that matters for your demo.

4. API key handling — what's already built in

Each feature calls its own key, read server-side only:

Feature	Key	Routes
1 · Coverage Check	<code>GEMINI_API_KEY_1</code>	<code>coverage-check</code> , <code>check-room</code> , <code>upload-report</code>
2 · Evidence Consistency	<code>GEMINI_API_KEY_2</code>	<code>evidence-check</code> , <code>room-walkthrough-check</code>
3 · Contradiction Flag	<code>GEMINI_API_KEY_3</code>	<code>contradiction-check</code>

If a key is missing or wrong in production, the app is designed to degrade gracefully rather than crash the page — the affected feature returns a clear "not configured" / error message instead of a broken screen. If you see that message live, it's almost always a typo'd variable name or a forgotten redeploy after adding it, not a code bug.

5. Troubleshooting quick reference

Symptom	Likely cause	Fix
Build fails on host, succeeds locally	Node version mismatch	Pin <code>"engines".node</code> in <code>package.json</code> to <code>≥20.9.0</code>

A feature shows "not configured"	Env var missing/misnamed, or not redeployed after adding it	Re-check spelling in dashboard, then trigger a redeploy
Load demo data / Upload report 500s only on the live site	Rare file-tracing edge case for fs reads under data/ or public/	Add outputFileTracingIncludes in next.config.mjs pointing at "data/**" and "public/sample-images/**"
Room Walkthrough or Load demo data times out	Multiple/parallel Gemini calls exceed the platform's function timeout	Add export const maxDuration = 60 to that route

Prepared for the Pillar To Post / PTP360 Second-Pass QA prototype. Reflects the codebase as of the deployment planning session — re-check the Step 5 smoke test after any future change to the data-reading routes (coverage-check, check-room, upload-report).